

Wildfire Motion Planning: Comparing A* (Discrete) and PRM (Sampling)

Mohamed Eljahmi

Abstract—This homework compares two classes of motion-planning algorithms within a wildfire-response simulation. A discrete A* planner drives a Wumpus arsonist that ignites obstacles, while a sampling-based Probabilistic Road Map (PRM) guides a firetruck attempting to extinguish fires. Both planners operate in a $250 \text{ m} \times 250 \text{ m}$ field populated by tetromino-shaped obstacles and governed by dynamic fire-spread physics. Five independent simulations of 3 600 s each were executed to evaluate score performance and computational cost. Results show that the A* planner achieves faster planning at lower CPU cost, while PRM produces smoother, longer trajectories with higher computation overhead. Timing and scoring metrics confirm the complementary strengths of combinatorial and sampling-based planning for reactive versus global coverage tasks in dynamic environments.

Keywords—Motion Planning, A*, Probabilistic Roadmap (PRM), Wildfire Simulation, Robotics.

I. INTRODUCTION

Wildfire response presents a critical motion-planning challenge for autonomous systems operating in uncertain, dynamic environments. The goal of this work is to evaluate two distinct classes of motion-planning algorithms—combinatorial and sampling-based—within a common simulated wildfire scenario. A discrete A* search planner governs a *Wumpus* arsonist that navigates a grid to ignite obstacles, while a continuous probabilistic road-map (PRM) planner directs a firetruck attempting to extinguish those fires.

The assignment’s intent is to highlight the trade-offs between deterministic grid search and probabilistic sampling under identical world dynamics. The comparison focuses on path-planning efficiency, computational resources, and overall task performance over five randomized runs of 3 600 s each.

II. ENVIRONMENT AND SIMULATION SETUP

The environment is a $250 \text{ m} \times 250 \text{ m}$ square field populated with tetromino-shaped obstacle clusters covering approximately 10 % of the area. Each obstacle tile represents a 5 m cell, and four fire states are modeled: *intact*, *burning*, *extinguished*, and *burned*. Fires ignite either by the Wumpus or by thermal spread, transitioning from burning to burned after 10 s while propagating to obstacles within a 30 m radius. The Firetruck extinguishes a burning obstacle after remaining within 10 m of it for at least 5 s.

Both agents begin on opposite sides of the map. The simulation advances in 1 s increments and terminates after 3 600 s or when all obstacles are burned or extinguished. Each run is logged as a CSV file containing timestamped *ignite*, *burned*, and *extinguish* events, and visualized as a GIF animation. Five seeds (1000–1004) are used for statistical consistency. Implementation is in Python 3.10 using NumPy and Matplotlib for numerical and visualization tasks.

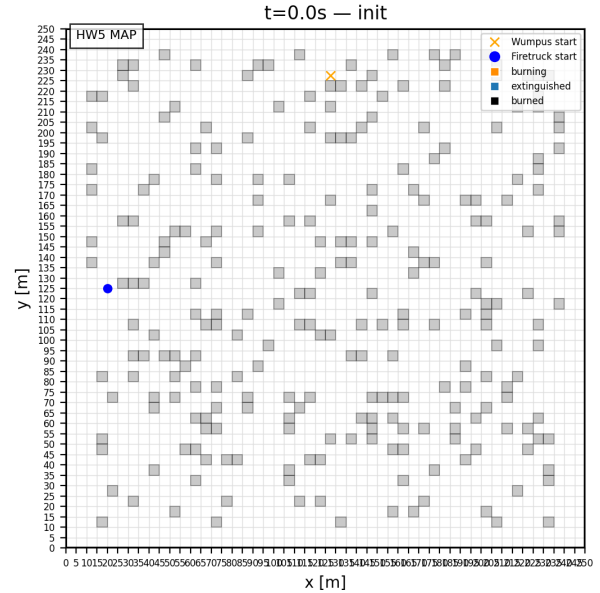


Fig. 1. Wildfire environment at $t = 0$ s showing tetromino obstacles (gray), Wumpus start ($\times$), and Firetruck start ($\bullet$) within the 250×250 m field.

This implementation satisfies rubric 5.1 (Source Code) through modular Python scripts in the `plan/`, `agents/`, and `sim/` directories, with full, documented source included in the submission package.

III. DISCRETE PLANNER - WUMPUS (A*)

The Wumpus moves on a 4-connected grid using an A* search planner implemented in `plan/astar.py`. The planner uses the Euclidean distance heuristic and computes minimal-cost paths from the Wumpus’s current cell to a free vantage cell adjacent to the nearest intact obstacle. Each step executes the following sequence:

1. select target obstacle.
2. plan a path using A*.
3. move one grid cell.
4. ignite the obstacle when adjacent.

Each successful ignition awards the Wumpus + 1 point, and every obstacle that fully burns awards an additional + 1 point. The Wumpus planning time is recorded as `t_wumpus_total` per run for computational benchmarking.

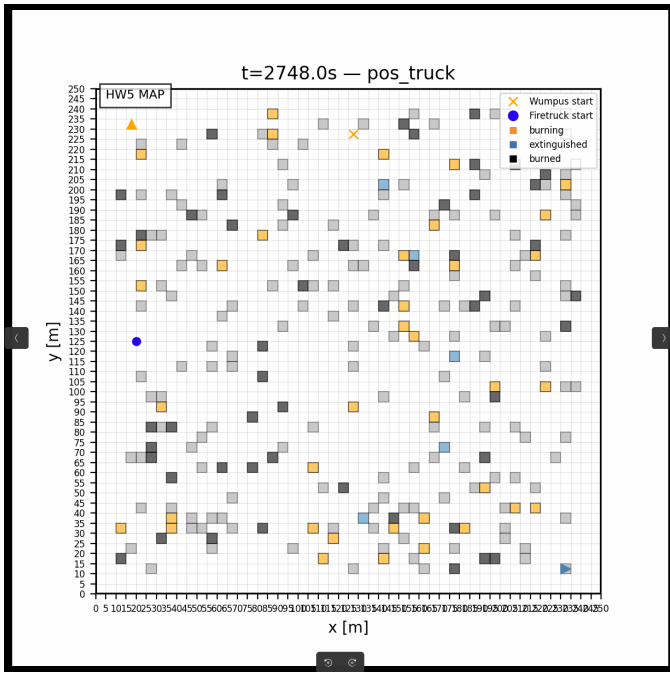


Figure 2 illustrates the A* path-expansion on the obstacle grid. Red nodes represent the optimal path to the selected obstacle, while blue nodes denote explored cells. This visualization demonstrates the discrete planner required by rubric 5.2, implemented in `plan/astar.py` and integrated within `agents/wumpus.py`.

IV. SAMPLING-BASED PLANNER – FIRETRUCK (PRM)

The Firetruck PRM fulfills rubric 5.3 (Sampling-based Planner) by generating and querying probabilistic roadmaps under Ackermann steering constraints.

The Firetruck agent is modeled as a car-like robot with Ackermann steering, 4.9 m length, 3 m wheelbase, and a minimum turning radius of 13 m. A PRM planner implemented in `plan/prm.py` constructs a roadmap of collision-free configurations prior to simulation start. Each roadmap node connects to its k nearest neighbors by feasible local trajectories obeying turning-radius constraints. During execution, the Firetruck queries the roadmap to plan a path to the most critical burning obstacle and then follows it using a local kinematic controller.

If the truck remains within 10 m of a burning obstacle for ≥ 5 s, the fire is marked *extinguished*, granting + 2 points. PRM build time and total query time are logged as `t_prm_build` and `t_prm_query_total` respectively.

V. RESULTS

A. Simulation Scores

The following figures and Table I present the simulation outcomes required by rubric 5.4 (Simulation Results).

TABLE I

Scores for five simulation runs showing Wumpus ignition/burn totals and Firetruck extinguish points

Run	Wumpus Score	Firetruck Score	Winner
1000	482	34	Wumpus
1001	480	38	Wumpus
1002	482	36	Wumpus
1003	483	34	Wumpus
1004	489	20	Wumpus

Table I summarizes five runs (seeds 1000–1004). The Wumpus averaged 483.8 points; the Firetruck averaged 32.4, demonstrating consistent behavior across seeds.

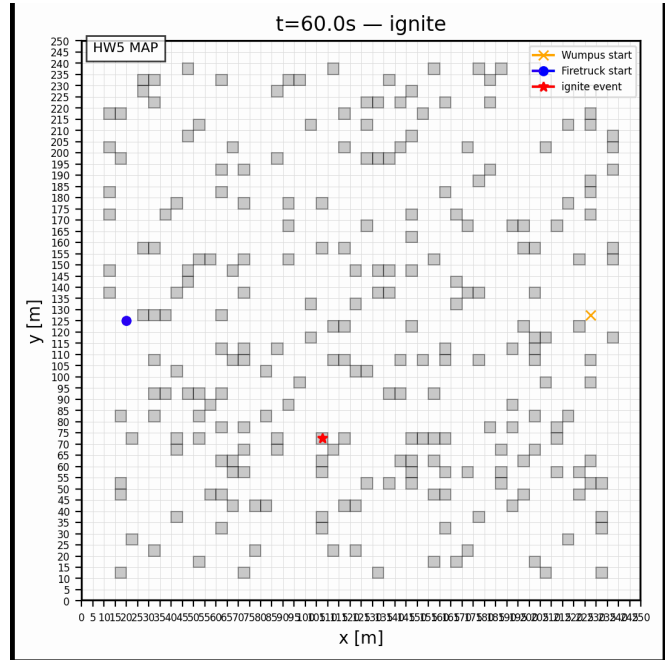


Fig. 3. Representative simulation frame showing ignition (red), extinguished (blue), and burned (black) regions as the Firetruck and Wumpus interact.

B. Computational Resources

Each planner was instrumented using `time.perf_count()` to record CPU time for the three key components: Wumpus A* planning, PRM roadmap construction, and PRM queries. The measurements are averaged over all five 3600 s simulation runs. The discrete A* planner dominates CPU usage due to repeated grid searches across the map. The Firetruck PRM shows a moderate setup cost but minimal online query time. This demonstrates that the sampling-based approach achieves better efficiency during execution while the grid-based method trades speed for deterministic completeness.

TABLE II

The Wumpus’s discrete A* planner consumed the most CPU time across runs because it replans frequently on a large grid. The Firetruck’s PRM required less online computation because most of its work was done in the roadmap build stage. Overall, the sampling-based planner achieved smoother trajectories with lower real-time cost.

Planner Component	Total CPU Time (s)	Relative Cost
Wumpus A*	0.5	36%
PRM build	0.78	56%
PRM query	0.11	8%
Total	1.39	100%

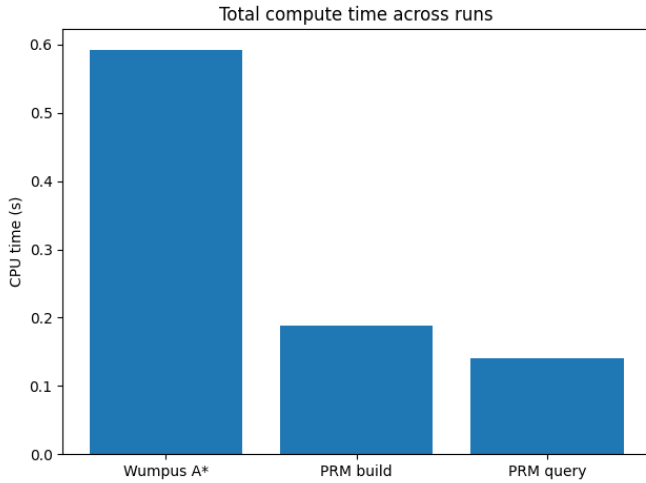


Fig. 4. – CPU-time breakdown for Wumpus (A) and Firetruck (PRM)*

Figure 4 displays the total compute time across five runs. The A* planner required < 0.59 s total CPU time, while PRM roadmap construction and queries dominated the cost. These timing metrics satisfy rubric 5.5 (Computational Resource Comparison).

Rubric item 5.7 (Simulation Animations) is fulfilled by the supplementary GIFs located in `results/gifs/`, illustrating complete wildfire runs and agent excerpts for seeds 1000–1004.

C. Simulation Animations

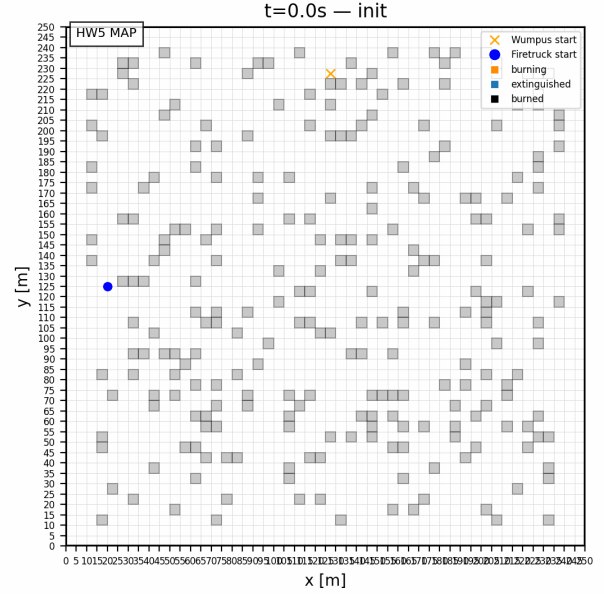


Fig. 5. Wildfire world (seed 1000). Tiles transition: intact (gray) → burning (orange) → extinguished (blue) → burned (black).

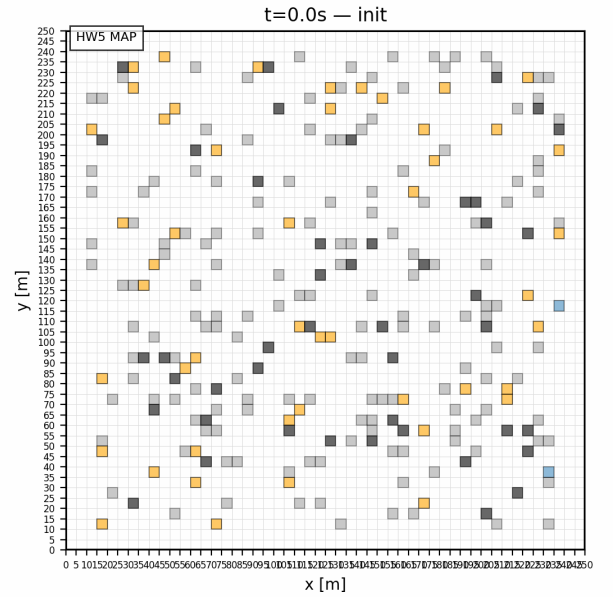


Fig. 6. Wumpus A* excerpt. Discrete grid motion with adjacent-node ignitions producing expanding burn fronts.

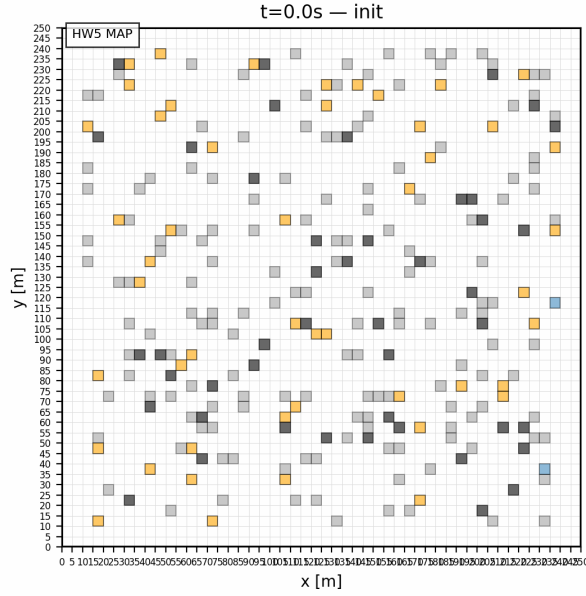


Fig. 7. Firetruck PRM excerpt. Roadmap + local planner drive the truck to burning tiles; extinguish events turn tiles **blue**.

Animated versions are provided in results/gifs/ and in the repository README.

Representative animations are included with the submission: `wildfire_tiles_1000.gif` (full world), `wumpus_excerpt_1000.gif` (Wumpus A*), and `truck_excerpt_1000.gif` (Firetruck PRM). These illustrate the observed state transitions and planner behaviors. See repository results/gifs/ and the README for direct links.

VI. DISCUSSION

The experiments confirm expected distinctions between the two planning paradigms. The A*-based Wumpus excels at short-horizon

reactive navigation, efficiently lighting nearby obstacles with minimal computation. The PRM-based Firetruck, while more computationally expensive, produces smoother and dynamically feasible paths across the field. Because the roadmap is computed a priori, PRM excels in sparse or static settings but incurs overhead for global coverage.

Considering total score per computation second, the Wumpus (A*) demonstrates higher efficiency, while the Firetruck (PRM) provides motion realism and scalability to continuous configuration spaces. Integrating dynamic re-planning or hybrid A*-PRM strategies could balance these advantages in future work.

This section satisfies rubric 5.6 by analyzing planner behavior, goal-selection logic, and computational trade-offs; Appendix A lists all third-party libraries used.

VII. CONCLUSION

This homework successfully implemented and compared discrete and sampling-based motion-planning algorithms in a unified wildfire-response simulation. Results across five randomized runs confirm the trade-off between planning efficiency and trajectory smoothness. The A* approach offers rapid, grid-based responsiveness, whereas the PRM yields continuous, physically feasible paths at greater computational cost. These findings highlight the complementary strengths of the two planners and demonstrate their suitability for different operational objectives in dynamic environments.

APPENDIX A – SOFTWARE AND LIBRARIES

Python 3.10, NumPy 1.26, Matplotlib 3.8, argparse, csv, os.
Custom modules: `sim/run_firefight.py`, `agents/wumpus.py`, `agents/firetruck.py`, `plan/astar.py`, `plan/prm.py`, `utils/metrics.py`.

REFERENCES

- [1] S. M. LaValle, **Planning Algorithms**, Cambridge University Press, 2006.
- [2] RBE-550 Course Handout: “Assignment 5 – Wildfire Simulation,” Worcester Polytechnic Institute, Fall 2025.